

# 智能生成系统开发课程笔记

## 一、开发环境

### 1. Anaconda

Anaconda 可以在以下系统平台中安装和使用：

- Windows
- macOS
- Linux

#### 1) 下载

前往官网 (<https://www.anaconda.com/download>)，登录（需先注册）后根据自己的操作系统选择安装程序，如图 1-1。其它系统可以通过点击相应操作系统的链接进行下载。

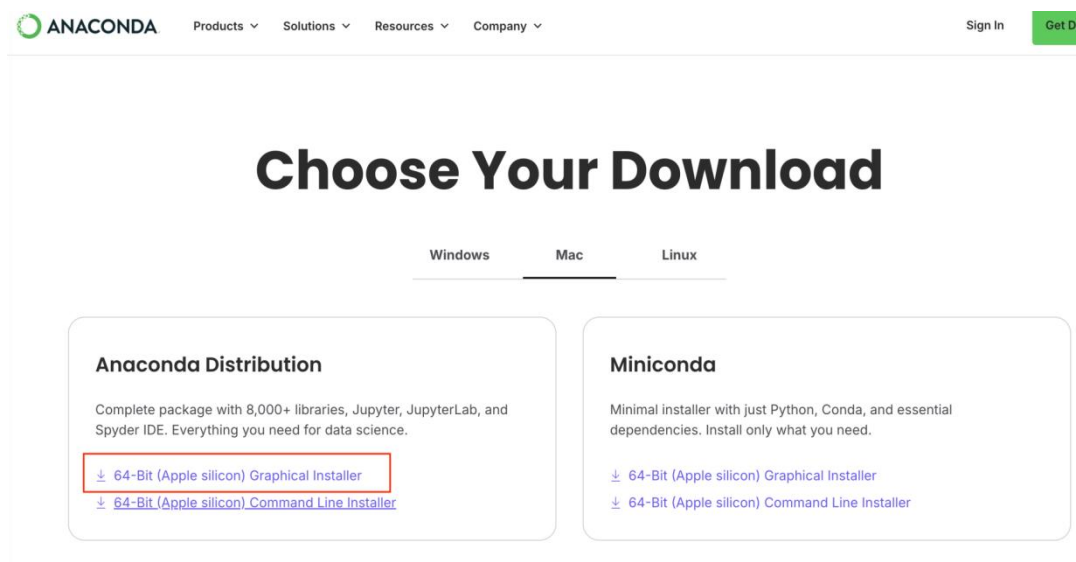


图 1-1 下载页面

## 2) 安装

下载完成之后，双击下载文件，在对话框中“介绍”、“请先阅读”、“许可”等步骤可直接点击下一步（或同意），如图 1-2。这里以 MacOS 系统为例，Windows 版本安装界面有所不同，但大同小异。

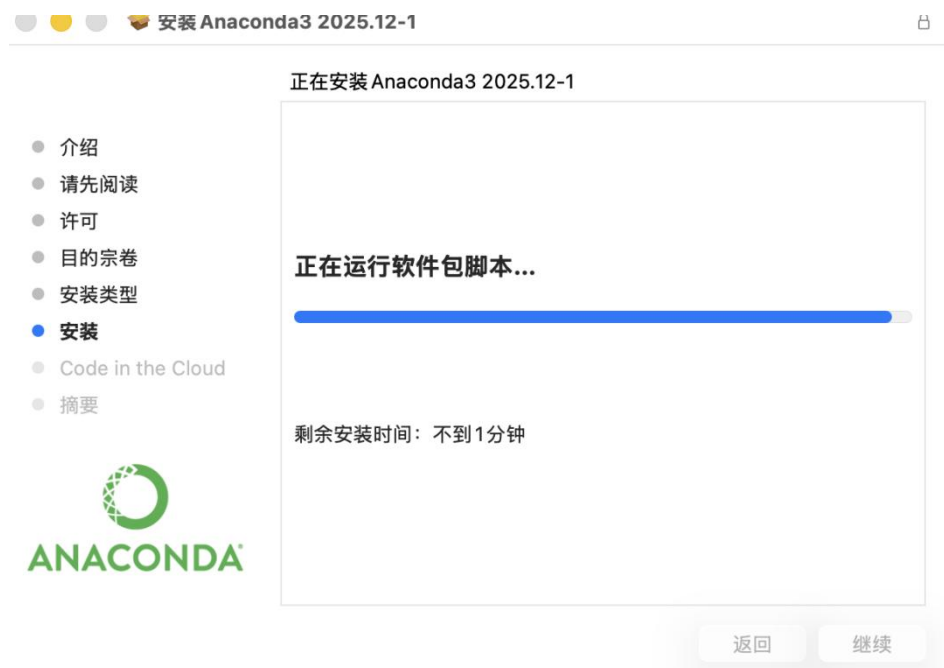


图 1-2 安装步骤 1

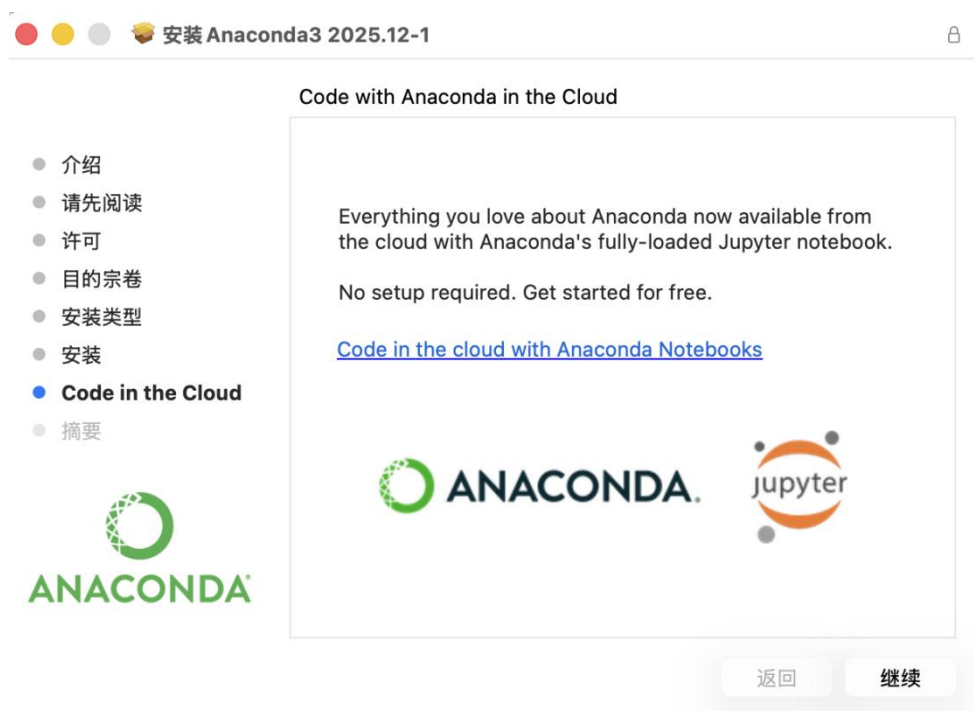


图 1-3 安装步骤 2

出现图 1-3 提示后点击“继续”即可进入“摘要”页，点击“关闭”后完成安装过程。

3) 使用

找到如图 1-4 所示应用程序图标, 启动 Anaconda Navigator (以下简称 AN)。



图 1-4 AN 程序图标

主界面如图 1-5，其中“Home”展现的是已经安装的工具（按钮是 Launch）和可以安装的工具（按钮是 Install）， 点击每个工具右上角的齿轮图标可以进行安装、卸载、升级等操作。

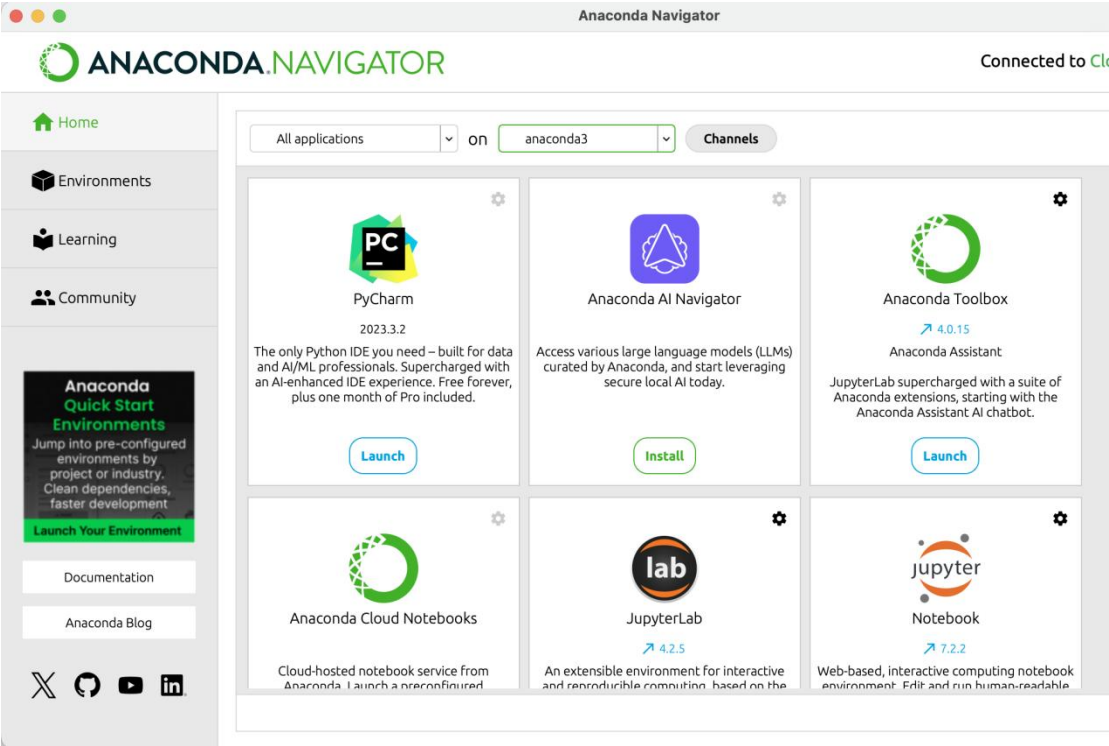


图 1-5 AN 主界面

“Environments” 为运行环境，默认只有一个 “base (root)”，可以通过底部按钮新建 (Create)、克隆 (Clone)、导入 (Import)、删除 (Remove)，如图 1-6。在以后的使用中，可以根据需要启动指定环境。最右边一列就是该环境下已经安装的库，如果版本号前面有一个箭头指示则表明该库有新版本可以升级。

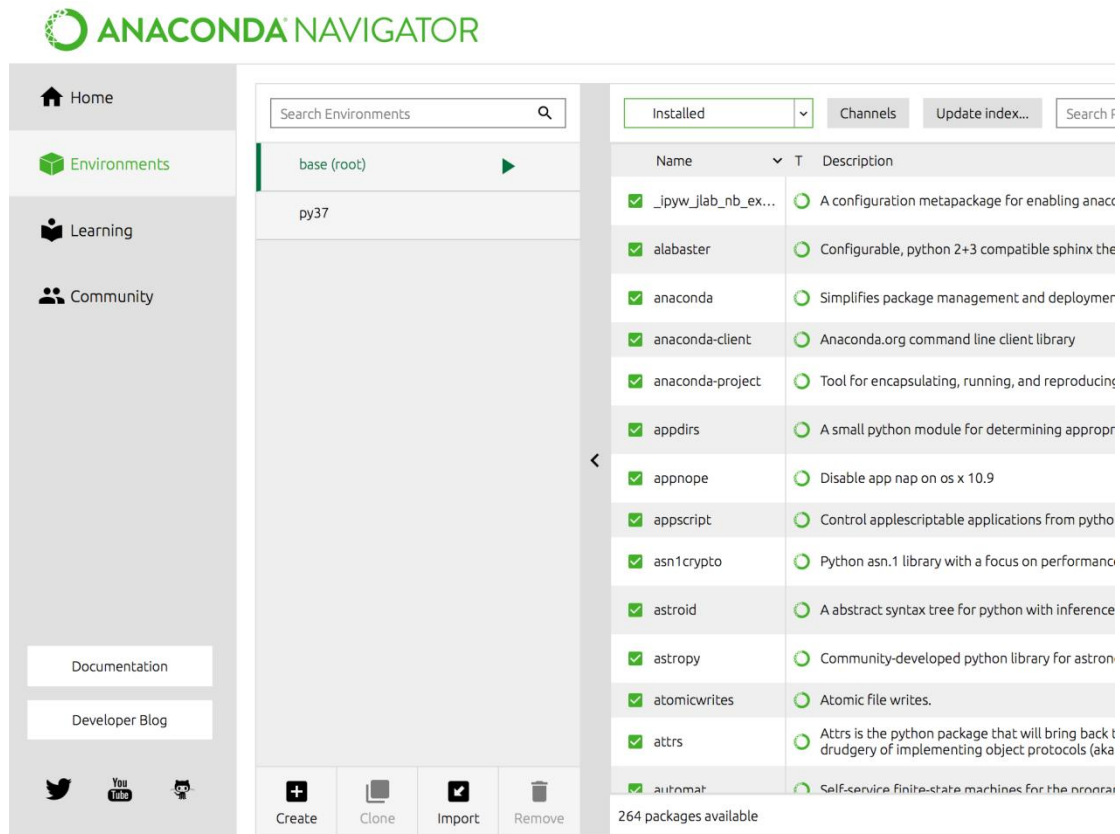


图 1-6 Environments 界面

#### 4) 安装和升级库

如果在 AN 中安装或升级库无反应，可以通过点击环境名后面的绿色三角后选择 “Open Terminal” 打开 conda 终端界面，如图 1-7。

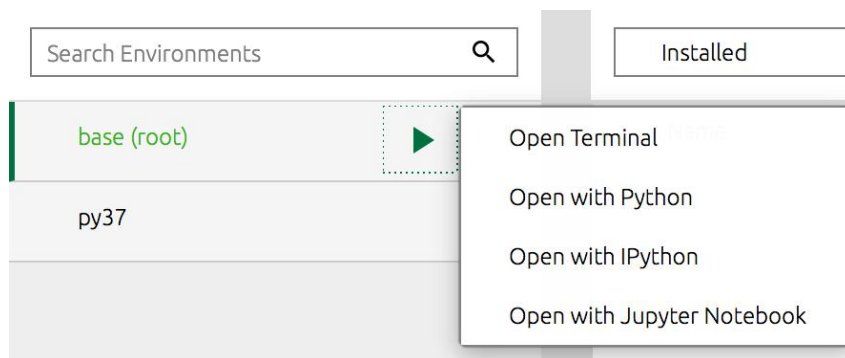


图 1-7 打开终端

在其中输入: `conda upgrade 库名称` 进行升级 (update 和 upgrade 相同), 如图 1-8。如果要安装新库, 则需输入: `conda install 库名称` 进行安装 (也可以使用 `pip install 库名称`)。

```
(base) bash-3.2$ conda upgrade appdirs
[Solving environment: done]

## Package Plan ##

environment location: /anaconda3

added / updated specs:
- appdirs

The following packages will be downloaded:
```

| package                      | build          |        |
|------------------------------|----------------|--------|
| conda-4.13.0                 | py37hecd8cb5_0 | 982 KB |
| openssl-1.0.2p               | h1de35cc_0     | 3.4 MB |
| certifi-2018.8.24            | py37_1         | 139 KB |
| ca-certificates-2018.03.07   | 0              | 124 KB |
| conda-package-handling-1.8.1 | py37hca72f7f_0 | 1.6 MB |
| appdirs-1.4.3                | py37h28b3542_0 | 16 KB  |
| Total:                       |                | 6.3 MB |

图 1-8 终端界面及升级命令

根据提示输入 y 开始升级, n 则取消升级, 如图 1-9。

```
Proceed ([y]/n)? y
```

```
Downloading and Extracting Packages
conda-4.13.0           | 982 KB      | #####
openssl-1.0.2p        | 3.4 MB      | #####
certifi-2018.8.24     | 139 KB      | #####
ca-certificates-2018  | 124 KB      | #####
conda-package-handli  | 1.6 MB      | #####
appdirs-1.4.3         | 16 KB       | #####
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
(base) bash-3.2$
```

图 1-9 终端升级过程

升级完成后回到 AN 图形界面，点击“Update index...”（图 1-6 上方）更新版本信息。

有些库无法直接安装，在安装时会报告如图 1-10 的信息。

```
[(py37) bash-3.2$ conda install opencv-python
Collecting package metadata (current_repodata.json): done
Solving environment: failed with initial frozen solve. Retrying w
lve.
Collecting package metadata (repodata.json): done
Solving environment: failed with initial frozen solve. Retrying w
lve.

PackagesNotFoundError: The following packages are not available f
nnels:

- opencv-python
```

图 1-10 安装时报告找不到的提示信息

此时可以去官网 [anaconda.org](https://anaconda.org) 搜索库名，如图 1-11；选择合适的检索结果，如图 1-12；再复制官网提供的安装命令，如图 1-13；最后在终端环境中粘贴命令进行安装。



Where packages, notebooks, projects and environments are shared.

SEARCH PACKAGES

图 1-11 搜索所需安装的库名

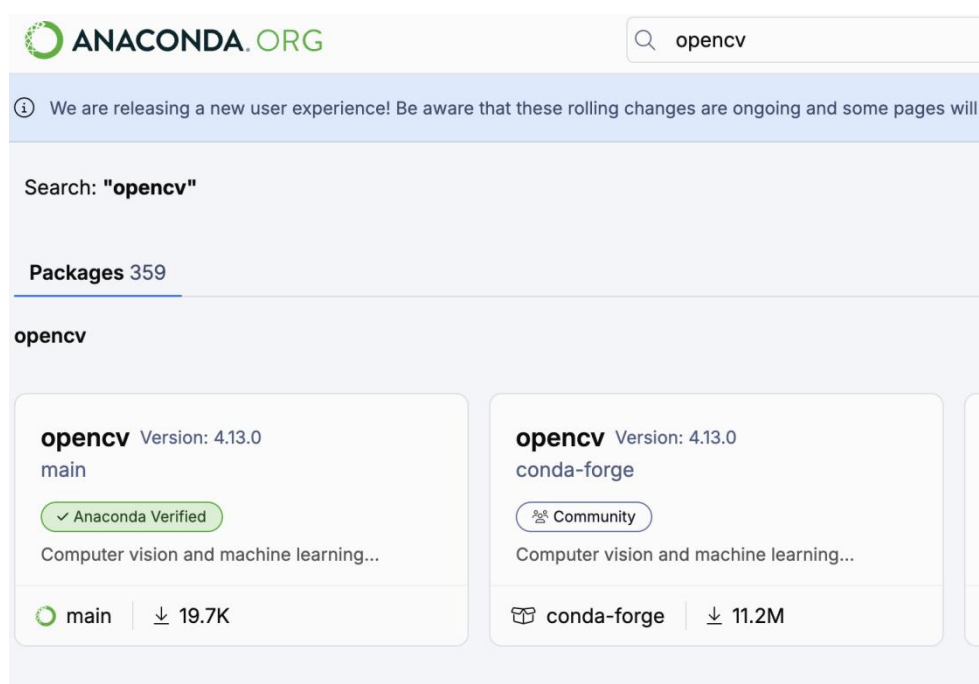


图 1-12 选择合适的搜索结果

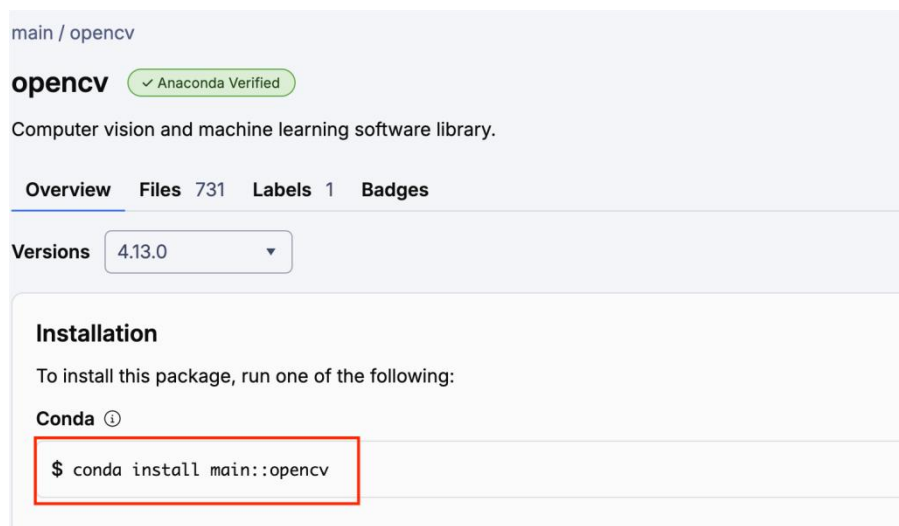


图 1-13 复制安装命令

其中由于国内限制，使用一些国内源速度会更快，常用的有清华大学（<https://pypi.tuna.tsinghua.edu.cn/simple/>）。安装时使用如下命令：

```
pip install package_name -i https://pypi.tuna.tsinghua.edu.cn/simple/
```

#其中 -i 为指定安装源

## 5) Jupyter Notebook 配置

JN 的配置文件位于 USER\_HOME 的 .jupyter 目录中，名为 jupyter\_notebook\_config.py，具体配置项及其含义可以参见官网，这里以 JN 中文件的保存路径为例进行说明。使用编辑器打开该文件，按照图 1-14 所示修改第 392 行 “=” 后面的路径即可。

```
390 ## 用于笔记本和内核的目录。
391 # Default: ''
392 c.NotebookApp.notebook_dir = '/'
```

图 1-14 JN 配置文件

## 2. PyCharm

### 1) 下载

前往官网 (<https://www.jetbrains.com/pycharm/download/>)，根据自己的操作系统选择最新版安装程序，如图 1-15。其它系统或版本可以通过点击页面相关链接获取。

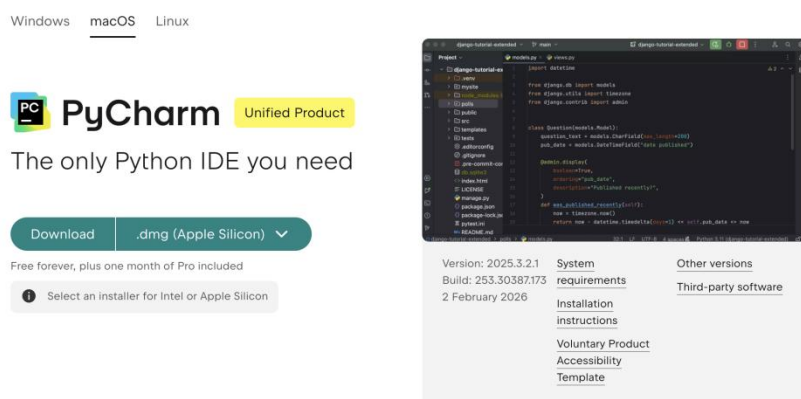


图 1-15 PyCharm 下载页面

### 2) 安装

目前 JetBrains 采用统一版本策略，前一个月开放 Pro 版功能，一个月后需要订阅，否则只能使用基础功能。如无特殊需求，则正常安装使用即可；如果需要 Pro 版功能，则可参考网站 (<https://www.exception.site/>) 给出的步骤进行安装破解，这里就不详细说明了。在校生可以通过申请获取免费使用 Pro 版功能的账号 (<https://sales.jetbrains.com/hc/zh-cn/articles/207154369>)。

### 3) PyCharm 配置 conda 虚拟环境

A. 已有项目修改，如图 1-16

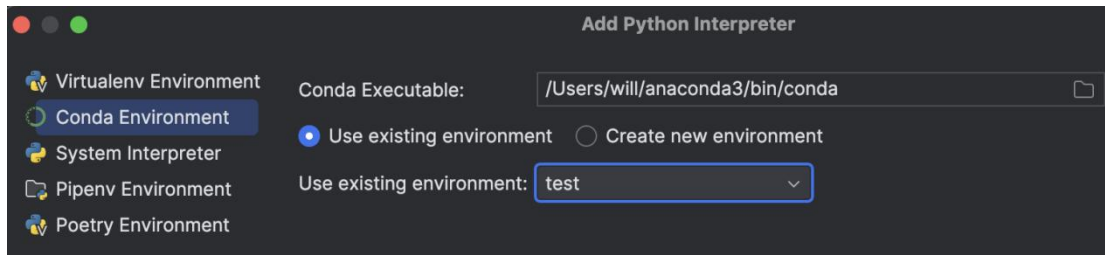


图 1-16 已有项目修改环境配置

## B. 新建项目设置，如图 1-17

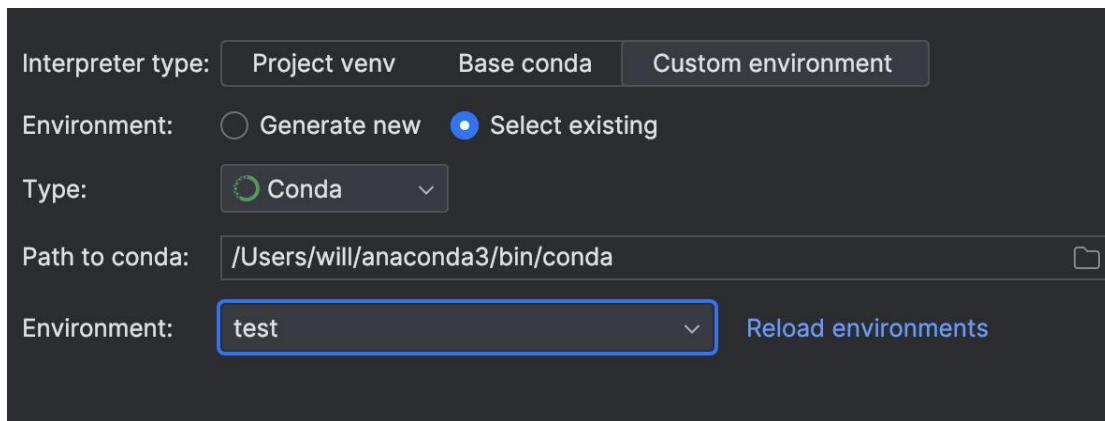


图 1-17 新建项目设置环境

## 3. Ollama

### 1) 介绍

Ollama 是一家专注于人工智能和机器学习的技术公司，致力于开发和提供领先的 AI 解决方案和工具。Ollama 提供一个简明易用的本地大模型运行框架，专注于开发和部署先进的大语言模型（LLM），只需一条命令即可在本地跑大模型。

Ollama 的优势包括：

- 易于使用: Ollama 提供了一个简单的 API，即使是没有经验的用户也可以轻松使用。
- 轻量级: Ollama 的代码简洁明了，运行时占用资源少。
- 可扩展: Ollama 支持多种模型架构，并可以扩展以支持新的模型。
- 预构建模型库: Ollama 提供了一个预构建模型库，可以用于各种任务，例如文本生成、翻译、问答等。

Ollama 的出现，使得在本地运行大型语言模型变得更加容易和方便。这对于以下用户群体来说非常有价值：

- 研究人员: Ollama 可以帮助研究人员快速原型化和测试新的 LLM 模型。
- 开发人员: Ollama 可以帮助开发人员将 LLM 功能集成到他们的应用程序中。
- 爱好者: Ollama 可以帮助爱好者探索和学习 LLM 技术。

## 2) 使用

A. 下载：访问官网（地址：<https://ollama.com/download>），根据自身操作系统下载相应的 Ollama 安装程序，如图 1-18。

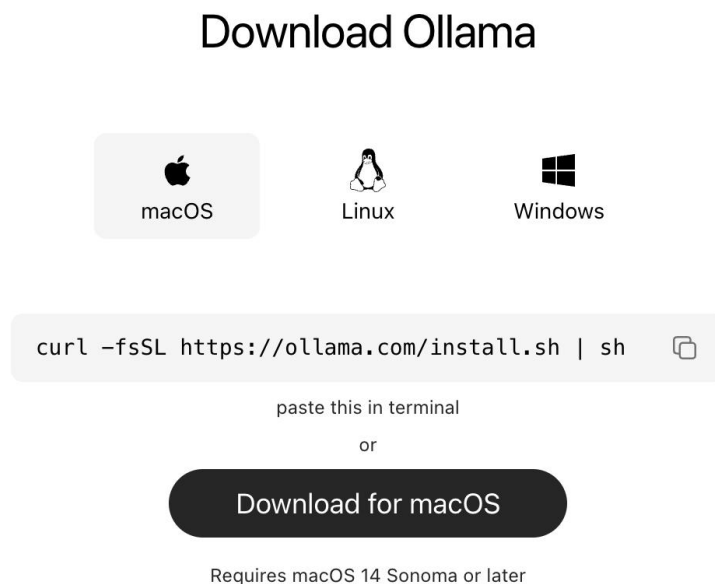


图 1-18 Ollama 下载页面

B. 安装：通过下载页面所提供的安装脚本进行安装或者下载安装程序进行安装。

C. 下载/运行模型：

通过命令“ollama run 模型名称”，可以直接运行指定模型，如果该模型本地不存在，则先到模型库中下载模型后运行。

具体可用模型，可通过官网模型库（<https://ollama.com/search>）搜索，如图 1-19。

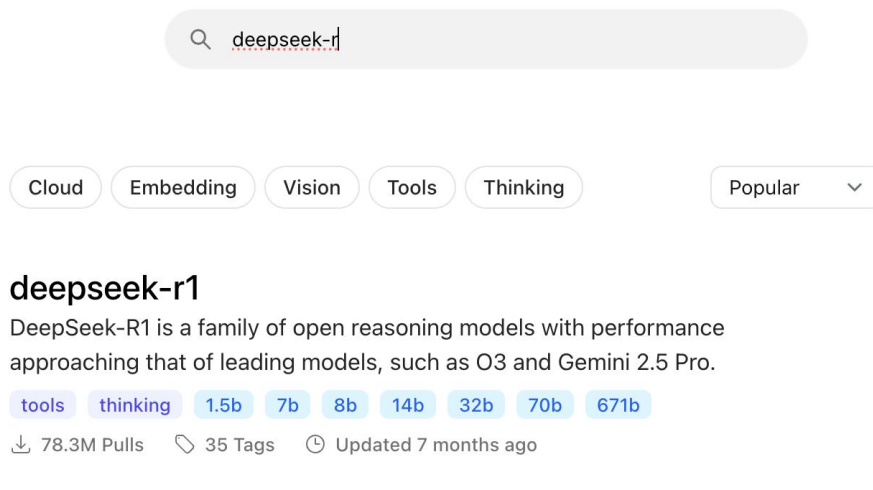


图 1-19 模型搜索

点击进入模型库后，根据自身硬件情况、应用目标选择不同（参数、类型）的版本，点击红框中的图标复制相应的脚本代码，如图 1-20。

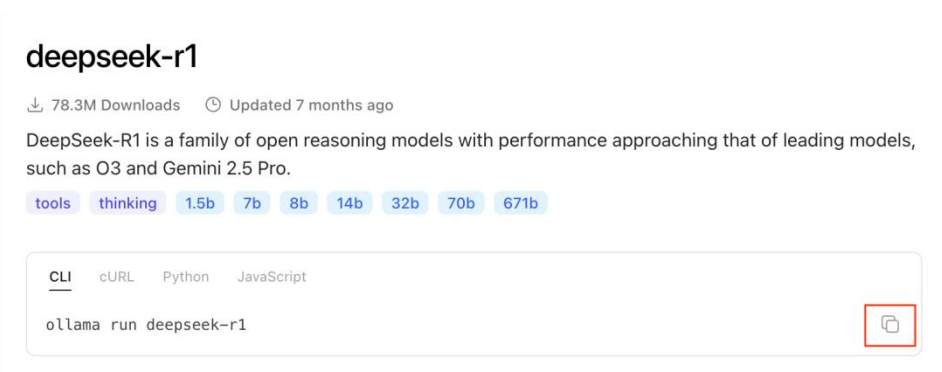


图 1-20 模型安装脚本

这里以运行 DeepSeek-R1 为例：ollama run deepseek-r1，如图 1-21（如果选择不同参数版本，需在模型名称后添加参数版本，如：ollama run deepseek-r1:1.5b）。

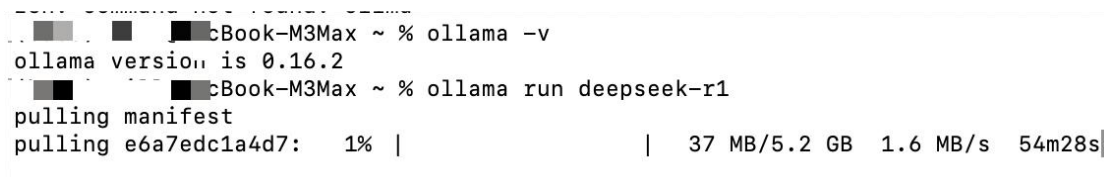


图 1-21 模型安装过程

运行成功后会进入终端聊天模式进行会话，图 1-22 中红框中即为输入内容。

```
writing manifest
success
[>>> please intro you]
Thinking...
Okay, the user just asked me to intro
straightforward request, but I should
```

图 1-22 聊天模式

### 3) Ollama 常用命令

- 启动 Ollama 服务: `ollama serve`
- 从模型文件创建模型: `ollama create`
- 显示模型信息: `ollama show`
- 运行模型: `ollama run 模型名称`
- 从模型库中拉取模型: `ollama pull 模型名称`
- 将模型推送到模型库中: `ollama push`
- 列出模型: `ollama list`
- 复制模型: `ollama cp`
- 删除模型: `ollama rm 模型名称`
- 获取有关 Ollama 任何命令的帮助信息: `ollama help`

## 4. Chatbox AI

### 1) 下载和安装

前往官网 (<https://chatboxai.app/zh/#download>)，根据自己的操作系统选择最新版安装程序，如图 1-23。其它系统或版本可以通过点击页面相关链接获取。



图 1-23 Chatbox AI 下载页面

## 2) 设置

打开 Chatbox，点击“设置”，在“模型提供方”里选择“Ollama”，保持“API 主机”不变（Ollama 默认使用本机 11434 端口），点击“获取”（在 Ollama 已经运行的前提下），选择大模型名称即可，如图 1-24。

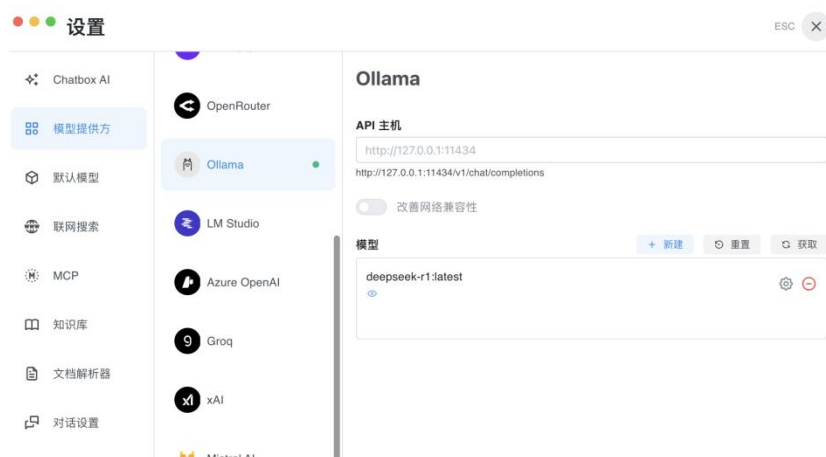


图 1-24 Chatbox AI 设置模型

另可点击大模型名称后面的齿轮图标，进行模型编辑，编辑完成后点击“测试模型”，显示测试成功后点击“保存”即可，如图 1-25。

编辑模型

×

模型ID

deepseek-r1:latest

显示名称

可选

模型类型

聊天

能力

☒ 视觉

☐ 推理

☐ 工具使用

高级设置

上下文窗口

最大输出Token数

例如 128000

例如 4096

测试成功

取消

测试模型

保存

图 1-25 Chatbox AI 模型编辑

## 二、实践操作

### 1. 访问本地 LLM

#### 1) 使用 Ollama 方式

A. 安装依赖: `pip install ollama`

B. 引入依赖: `import ollama`

C. 指定模型、消息等创建会话

```
response = ollama.chat(  
    model='deepseek-r1:latest',  
    messages=[  
        {'role': 'user', 'content': '你好, 请介绍一下你自己'}  
    ]  
)
```

D. 打印响应信息

```
print(response['message']['content'])
```

#### 2) 使用 OpenAI 方式

A. 安装依赖: `pip install openai`

B. 引入依赖: `from openai import OpenAI`

C. 创建指向 Ollama 服务的客户端

```
client = OpenAI(  
    base_url='http://localhost:11434/v1', # Ollama 默认地址  
    api_key='ollama' # 本地模式不需要真实 key, 但不能为空  
)
```

D. 创建聊天会话

# 指定模型、消息、温度、最大长度等

```
response = client.chat.completions.create(  

```

```

model='deepseek-r1:latest', # 你的模型名称
messages=[
    {'role': 'system', 'content': '你是一个有帮助的助手'},
    {'role': 'user', 'content': '解释一下量子计算的基本原理'}
],
temperature=0.7,
max_tokens=5000
)

```

#### E. 打印响应信息

```
print(response.choices[0].message.content)
```

## 2. 使用 LlamaIndex 嵌入本地数据（无向量数据库）

### 1) 安装 llama-index 依赖

```
pip install llama-index
```

注意: llama-index 的默认嵌入模型和生成模型都是 OpenAI, 需要 OpenAI 的 API Key, 因此这里使用 Ollama 部署的本地 DeepSeek 作为生成模型; 暂时不嵌入本地数据, 改用非向量检索

### 2) 安装 llama-index 的 Ollama 依赖

```
pip install llama-index-llms-ollama
```

注意: 以上依赖的版本更新较快, 常会因为出现不兼容而报错的情况, 请务必确认版本 (conda list 或 pip show 库名), 这里 llama-index==0.14.15, llama-index-llms-ollama==0.9.1

### 3) 引入相关依赖

```

from llama_index.core import SimpleDirectoryReader, Settings
from llama_index.core.indices.keyword_table import KeywordTableIndex
from llama_index.llms.ollama import Ollama

```

注意: from llama\_index.core import VectorStoreIndex 是引入向量存储,

KeywordTableIndex 为非向量存储

#### 4) 设置本地模型

```
Settings.llm = Ollama(  
    model="deepseek-r1:latest",  
    base_url="http://localhost:11434", #默认, 可省略  
    temperature=0.7,  
    request_timeout=120.0 #默认: 30 秒  
)
```

#### 5) 加载本地文件

```
documents = SimpleDirectoryReader(input_files=["./黑神话/黑悟空  
wiki.txt"]).load_data()
```

注意: 参数 `input_dir` 用于指定加载文件所在的目录; 参数 `recursive` 指定是否递归加载子目录中的文件, 参见:

[https://developers.llamaindex.ai/python/framework/module\\_guides/loading/simplydirectoryreader/](https://developers.llamaindex.ai/python/framework/module_guides/loading/simplydirectoryreader/)

#### 6) 不使用向量数据库建立索引 (比较耗费系统资源)

```
index = KeywordTableIndex.from_documents(documents)
```

#### 7) 创建查询引擎

```
query_engine = index.as_query_engine()
```

#### 8) 执行查询

```
response = query_engine.query("介绍一下黑神话悟空")
```

#### 9) 打印响应信息

```
print(response)
```